

CONVERSATIONCHAIN – STUDENT NOTES

WHAT IS CONVERSATIONCHAIN?

ConversationChain is an older abstraction in LangChain.

It automatically:

- Loads previous conversation history
- Uses it with current input
- Saves new interaction

STRUCTURE

Model
+
ConversationBufferMemory
↓
ConversationChain

CREATING A LEGACY CONVERSATIONCHAIN

Import:

```
from langchain_classic.chains import ConversationChain
from langchain_classic.memory import ConversationBufferMemory
```

Create memory:

```
memory = ConversationBufferMemory()
```

Create chain:

```
conversation = ConversationChain(
    llm = llm,
    memory = memory
)
```

CALLING THE CONVERSATIONCHAIN

Example:

```
result = conversation.invoke(  
  {  
    "input": "My name is Durga."  
  }  
)
```

PROGRAM 5 – CONVERSATIONCHAIN WITH BUFFER MEMORY

Steps:

1. Create model
2. Create memory
3. Create conversation chain

First call:

Input: My name is Durga
→ Stored in memory

Second call:

Input: What is my name?
→ Uses memory and answers correctly

KEY IDEA

ConversationBufferMemory stores full conversation history.

So model remembers all previous interactions.

PROGRAM 6 – WINDOW MEMORY

Instead of storing full history, we store limited messages.

Memory:

ConversationBufferWindowMemory(k = 2)

WHAT DOES k=2 MEAN?

Only last 2 interactions are stored.

Older messages are removed.

FLOW

1. My name is Durga
2. I teach Python
3. I live in Hyderabad

Now only last 2 are remembered:

- I teach Python
- I live in Hyderabad

FINAL QUESTION

Input: Where do I live?

Output: Hyderabad

ADVANTAGE

- Saves memory
- Faster execution
- Useful for long conversations

PROGRAM 7 – INTERACTIVE CHATBOT

Steps:

1. Create model

2. Create memory
3. Create ConversationChain

CHAT LOOP

- User gives input
- Model responds
- Conversation continues until user types "exit"

FEATURE

ConversationChain automatically remembers full conversation.

MESSAGESPLACEHOLDER – STUDENT NOTES

WHAT IS MESSAGESPLACEHOLDER?

MessagesPlaceholder is used to insert chat history into a prompt.

USAGE

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

Create prompt:

```
prompt = ChatPromptTemplate.from_messages([  
    ("system", "You are a helpful assistant."),  
    MessagesPlaceholder("history")  
])
```

EXAMPLE HISTORY

```
history = [  
    ("human", "My name is Durga."),
```

```
("ai", "Hello Durga!")  
]
```

INVOKE PROMPT

```
result = prompt.invoke({  
  "history": history  
})
```

OUTPUT

The history messages are inserted into the prompt.

Each message is treated as:

- Human message
- AI message

KEY POINTS

- ConversationChain = old method for chat memory
- Automatically handles history
- Buffer memory → stores full chat
- Window memory → stores limited chat
- MessagesPlaceholder → injects history into prompts

FINAL SUMMARY

- Use ConversationChain for simple chat applications
- Use memory to maintain context
- Use window memory for optimization
- Use MessagesPlaceholder to manually control history